COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010/ Data Structure and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	July 9 2026	**DATE SUBMITTED:**	July 14 2026
NAME:	Regala, Joaquin Luiz B.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. Hands-on 1.1 — **[ACTIVITY NAME]: Basic C++ Programming**

1. PROCEDURE OUTPUT

ILO A: Create Code That Follows the Basic C++ Code Structure

For this activity, you have to demonstrate the use of a **function prototype**. The section on class declaration and method definition will be used for the function prototype and the function will be defined in the follow method definition section after the main function.

A function prototype in c++ is a declaration of the name, parameters and return type of the function before its definition. Write a C++ code the satisfies the following:

- Create a function that will take two numbers and display the sum.
- Create a function that will return whether variable A is greater than variable B.
- Create a function that will take two Boolean values and display the result of all logical operations then return true if it was a success.

SCREENSHOTS:

```cpp
#include <iostream>

void displaySum(int num1, int num2);
bool isGreater(int a, int b);
bool logicalOperations(bool x, bool y);

int main() {
    int num1, num2;
    int a, b;
    bool x, y;

    std::cout << "Enter two numbers: ";
    std::cin >> num1 >> num2;
    displaySum(num1, num2);

    std::cout << "\nEnter two numbers to compare: ";
    std::cin >> a >> b;

    if (isGreater(a, b)) {
        std::cout << "A is greater than B." << std::endl;
    } else {
        std::cout << "A is not greater than B." << std::endl;
    }
```

```
    std::cout << "\nEnter two boolean values (0 for false, 1 for true): ";
    std::cin >> x >> y;

    if (logicalOperations(x, y)) {
        std::cout << "Function executed successfully." << std::endl;
    }

    return 0;
}


void displaySum(int num1, int num2) {
    std::cout << "Sum = " << num1 + num2 << std::endl;
}

bool isGreater(int a, int b) {
    return a > b;
}

bool logicalOperations(bool x, bool y) {
    std::cout << "AND: " << (x && y) << std::endl;
    std::cout << "OR: " << (x || y) << std::endl;
    std::cout << "NOT x: " << (!x) << std::endl;
    std::cout << "NOT y: " << (!y) << std::endl;
    return true;
}


//CODE ANALYSIS
//This code shows the use of function prototypes by declaring the functions before the main() and defining them afterward.
//It has three functions one that displays the sum of two numbers, one that checks if the first number is greater than the second,
//and one that performs basic logical operations on two Boolean values.
//The main() function gathers user input and calls each function to perform its assigned task.
//The program follows the basic C++ code structure while keeping the functions organized and easy to understand.
```

ILO B: Implement Appropriate Class Definition and Instances Based on Given Requirements

In this section, the initial implementation for a class **triangle** will be implemented. The step-by-step procedure is shown below:

SECTIONS:	ANSWER:
Header File Declaration Section	`#include <iostream>`
Global Declaration Section	none
Class Declaration and Method Definition Section	`class Triangle{ private: double totalAngle, angleA, angleB, angleC; public: Triangle(double A, double B, double C); void setAngles(double A, double B, double C); const bool validateTriangle(); };`
Main Function	`int main(){ Triangle set1(40,30,20); if(set1.validateTriangle()){ std::cout<<"The shape is a valid Triangle"<<std::endl; } else{ std::cout<<"The shape is NOT a valid Triangle. \n"; } return 0; }`

Method Definition	

```cpp
Triangle::Triangle(double A, double B, double C){
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A+B+C;
}

void Triangle::setAngles(double A, double B, double C){
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A+B+C;
}

const bool Triangle::validateTriangle(){
    return (totalAngle <= 180);
}
```

Screenshot:

```cpp
#include <iostream>


class Triangle{
    private:
        double totalAngle, angleA, angleB, angleC;
    public:
    Triangle(double A, double B, double C);

    void setAngles(double A, double B, double C);

    const bool validateTriangle();
};

int main(){

    Triangle set1(40,30,20);
    if(set1.validateTriangle()){
        std::cout<<"The shape is a valid Triangle"<<std::endl;
    }   else{
        std::cout<<"The shape is NOT a valid Triangle. \n";
    }

    return 0;
}

Triangle::Triangle(double A, double B, double C){
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A+B+C;
}

void Triangle::setAngles(double A, double B, double C){
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A+B+C;
}

const bool Triangle::validateTriangle(){
    return (totalAngle <= 180);
}
```

Observation and Comments:
My observation is that the code demonstrates a good use of functions by organizing the triangle data and functions inside the triangle class. The setAngles function properly initializes the angle values while the validateTriangle function checks if it is a valid or a NOT valid triangle. The validation checks if the total angle meets the exact 180 degree instead of less than or equal to 180 for an accurate result.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve Different Problems using the C++ Programming Language

The supplementary activities are meant to gauge your ability in using C++. The problems below range from easy to intermediate to advanced problems. Note your difficulties after answering the problems below.

1. Create a C++ program to swap the two numbers in different variables.
2. Create a C++ program that has a function to convert temperature in Kelvin to Fahrenheit.
3. Create a C++ program that has a function that will calculate the distance between two points.
4. Modify the code given in ILO B and add the following functions:
    a. A function to compute for the area of a triangle
    b. A function to compute for the perimeter of a triangle
    c. A function that determines whether the triangle is acute-angled, obtuse-angled or 'others.'

1.

```cpp
#include <iostream>

void swapNumbers(int &a, int &b);

int main()
{
    int num1, num2;

    std::cout << "Enter first number: ";
    std::cin >> num1;

    std::cout << "Enter second number: ";
    std::cin >> num2;

    std::cout << "\nBefore swapping:" << std::endl;
    std::cout << "First Number: " << num1 << std::endl;
    std::cout << "Second Number: " << num2 << std::endl;

    swapNumbers(num1, num2);

    std::cout << "\nAfter swapping:" << std::endl;
    std::cout << "First Number: " << num1 << std::endl;
    std::cout << "Second Number: " << num2 << std::endl;

    return 0;
}

void swapNumbers(int &a, int &b)
{
    int temp;

    temp = a;
    a = b;
    b = temp;
}
```

```
Enter first number: 11
Enter second number: 7

Before swapping:
First Number: 11
Second Number: 7

After swapping:
First Number: 7
Second Number: 11


=== Code Execution Successful ===
```

2.

```cpp
#include <iostream>

// Function Prototype
double kelvinToFahrenheit(double kelvin);

int main()
{
    double kelvin, fahrenheit;

    std::cout << "Enter temperature in Kelvin: ";
    std::cin >> kelvin;

    fahrenheit = kelvinToFahrenheit(kelvin);

    std::cout << "Temperature in Fahrenheit: " << fahrenheit << std::endl;

    return 0;
}

// Function Definition
double kelvinToFahrenheit(double kelvin)
{
    return (kelvin - 273.15) * 9 / 5 + 32;
}
```

```
Enter temperature in Kelvin: 2700
Temperature in Fahrenheit: 4400.33


=== Code Execution Successful ===
```

3.

```cpp
#include <iostream>
#include <cmath>

double calculateDistance(double x1, double y1, double x2, double y2);

int main()
{
    double x1, y1, x2, y2, distance;

    std::cout << "Enter the coordinates of the first point (x1 y1): ";
    std::cin >> x1 >> y1;

    std::cout << "Enter the coordinates of the second point (x2 y2): ";
    std::cin >> x2 >> y2;

    distance = calculateDistance(x1, y1, x2, y2);

    std::cout << "Distance between the two points: " << distance << std::endl;

    return 0;
}

double calculateDistance(double x1, double y1, double x2, double y2)
{
    return std::sqrt(std::pow(x2 - x1, 2) + std::pow(y2 - y1, 2));
}
```

```
Enter the coordinates of the first point (x1 y1): 360
110
Enter the coordinates of the second point (x2 y2):
70

15
Distance between the two points: 305.164


=== Code Execution Successful ===
```

4.

```cpp
#include <iostream>
#include <cmath>

class Triangle
{
private:
    double sideA, sideB, sideC;
    double totalAngle, angleA, angleB, angleC;

public:
    Triangle(double A, double B, double C, double sA, double sB, double sC);

    void setAngles(double A, double B, double C);

    bool validateTriangle();

    double perimeter();

    double area();

    void triangleType();
};

int main()
{
    Triangle set1(60, 60, 60, 5, 5, 5);

    if (set1.validateTriangle())
    {
        std::cout << "The shape is a valid Triangle." << std::endl;

        std::cout << "Perimeter: " << set1.perimeter() << std::endl;
        std::cout << "Area: " << set1.area() << std::endl;

        set1.triangleType();
    }
    else
    {
        std::cout << "The shape is NOT a valid Triangle." << std::endl;
    }

    return 0;
}
```

```cpp
Triangle::Triangle(double A, double B, double C, double sA, double sB, double sC)
{
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A + B + C;

    sideA = sA;
    sideB = sB;
    sideC = sC;
}

void Triangle::setAngles(double A, double B, double C)
{
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A + B + C;
}

bool Triangle::validateTriangle()
{
    return (totalAngle == 180);
}


double Triangle::perimeter()
{
    return sideA + sideB + sideC;
}

double Triangle::area()
{
    double s = perimeter() / 2;

    return sqrt(s * (s - sideA) * (s - sideB) * (s - sideC));
}
```

```
82
83  void Triangle::triangleType()
84  {
85      if (angleA < 90 && angleB < 90 && angleC < 90)
86      {
87          std::cout << "Triangle Type: Acute-angled" << std::endl;
88      }
89      else if (angleA > 90 || angleB > 90 || angleC > 90)
90      {
91          std::cout << "Triangle Type: Obtuse-angled" << std::endl;
92      }
93      else
94      {
95          std::cout << "Triangle Type: Others (Right Triangle)" << std::endl;
96      }
97  }
```

```
The shape is a valid Triangle.
Perimeter: 15
Area: 10.8253
Triangle Type: Acute-angled


=== Code Execution Successful ===
```

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Chatgpt**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar/spelling fix and tips on how to fix errors in code**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Chatgpt was used to fix my spelling and grammar errors in the paragraphs and sentences, and also to help on some code assistance with the errors**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.